

Lab 4: Playing around with Pointers

- 1. Identify and explain which of the following expressions are valid and which are not valid as shown with an arrow. 5 pts**

char c;

char *ptr;

int f;

ptr = &c; ← This is valid, it will set the char pointer to the memory address of c.

ptr = &f; ← This is not valid, you will receive a warning about invalid pointer assignment because you are assigning a char pointer to an int value.

ptr = &'#'; ← This is not valid, # is not a valid variable

ptr = &500; ← This is not valid, the scalar 500 has no memory address

ptr = &(f+3); ← This is not valid, cannot increment the memory address of an integer. That is array pointer notation.

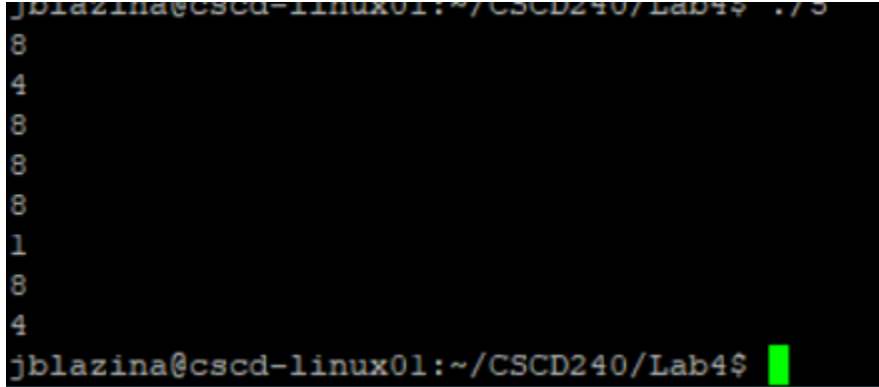
- 2. What will be the size of the following pointers? What are their scalar values? Explain with screen shots. 4 pts**

int * p; Size is 8 bytes. Scalar value is 4 bytes

double *q; Size is 8 **bytes**. Scalar size is 8 **bytes**

char* r; Size is 8 **bytes**. Scalar size is 1 **bytes**

float* s; Size is 8 **bytes**. Scalar size is 4 **bytes**

A terminal window with a black background and yellow text. The prompt is 'jblazina@cscd-linux01:~/CSCD240/Lab4\$'. The user has entered 'sizeof' followed by several lines of output: '8', '4', '8', '8', '8', '1', '8', '4'. The prompt is now 'jblazina@cscd-linux01:~/CSCD240/Lab4\$' with a green cursor.

```
jblazina@cscd-linux01:~/CSCD240/Lab4$ sizeof
8
4
8
8
8
1
8
4
jblazina@cscd-linux01:~/CSCD240/Lab4$
```

3. What is wrong with the following program? Please explain. How will you fix it? 1 pt

```
#include <stdio.h>
int main(){
    int i;
    int *ptr = &i;
    scanf("%d", &ptr); //change to -> scanf("%d", ptr);
    printf("The value of i is: %d\n", *ptr);
    return 0;
}
```

You cannot use the pointer's address to input values to. You have to remove the & from ptr in the scanf function for it to work properly.

4. What will be the values of the variables in the lines marked with arrows.

NOTE:

Less ambiguous phrasing suggestion. Since she can't yell at me here.

What will be printed by:

```
printf("%d", c = *p);  
printf("%d", *p = *p * *p);  
printf("%d", (*p)++);  
printf("%d", c = *&a);
```

```
int c, a = 10;
```

4 pts

```
int *p = &a;
```

`c = *p;` $\leftarrow$ `c = 10` due to it dereferencing the pointer that is pointing to the memory address of `a`, which is 10.

`*p = *p * *p;` $\leftarrow$ the value at the memory address pointed to by `p` will be multiplied by itself. The new value will be 100 (10×10)

`(*p)++;` $\leftarrow$ Will print 100, because of the postfix increment.

`c = *&a;` $\leftarrow$ Sets `c` to be the dereferenced value at `a`, so `c` will now also equal 101.

5. Consider the following piece of code. What will be printed by `*ptr` and `ptr` after the lines shown with an arrow? Explain with screenshots.

```
int a[4] = { 8, 3, 5, 6};
```

4 pts

`int *ptr = a;` $\leftarrow$ `*ptr` will print 8, because it is the first value of the array. `ptr` will print the memory address of `a`, which is the same as the memory address of `a[0]`.

`ptr ++;` $\leftarrow$ `*ptr` will print 3, because we incremented the pointer by one position. `Ptr` will print the memory address of `a[1]`.

```
jblazina@cscd-linux01:~/CSCD240/Lab4$ ./5
8
0x7ffee927b180
3
0x7ffee927b184
jblazina@cscd-linux01:~/CSCD240/Lab4$
```

6. What is the difference between the following two declarations? 2 pts

`char array[] = "Hello World";`

`char *array = "Hello World";`

The two declarations are almost the same, a string is just a character array so the second one is a pointer to a character array, while the first one is an explicit declaration of a character array which works the same way. The difference is that the `char *` declaration makes it immutable while the `array[]` declaration allows it to be mutable.

Submission:

A pdf file containing answers to the questions and output capture wherever necessary.

Name your file with your last name first letter of your first name Lab4.pdf (ex: yasminsLab4.pdf).

Submission deadline is: 11:59 pm, Monday, February 11. No late submission will be considered.